

ENTSCHEIDUNGSGRUNDLAGE

Jama MCP im Vergleich

Offizieller Jama Connect MCP gegenüber der Eigenentwicklung

Version	1.0
Datum	2026-08-12
Status	Entscheidungsgrundlage
Gegenstand	Jama Connect MCP (offiziell) gegenüber jama-mcp-service (Eigenentwicklung)

Inhalt

Zusammenfassung

Rahmenbedingungen

Funktionsumfang im Überblick

Abdeckung der offiziellen Tools

Die 38 zusätzlichen Tools

Kern — 7 Tools

Traceability — 4 Tools

Schreibende Operationen — 7 Tools

Zusammenarbeit — 1 Tool

Test-Management — 7 Tools

Historie — 4 Tools

Reviews — 3 Tools

Dateien und Reports — 5 Tools

Dazu ohne Entsprechung

Was der Eigenbau darüber hinaus leistet

Von Jama ausdrücklich ausgeschlossene Fähigkeiten

Aufbereitung der Antworten

Betrieb und Steuerung

Was für den offiziellen Server spricht

Offene Risiken des Eigenbaus

Einschätzung

Nächste Schritte

Zusammenfassung

Beide Lösungen verbinden MCP-fähige Clients mit Jama Connect. Sie sind aber keine Alternativen im engeren Sinn, weil sie unterschiedliche Probleme lösen.

Der **offizielle Jama Connect MCP** ist ein von Jama betriebener Dienst mit 18 Tools. Er ist auf das gesicherte Abrufen und Bearbeiten einzelner Objekte ausgelegt und schließt Massenoperationen ausdrücklich aus.

Der **eigene jama-mcp-service** ist ein selbst betriebener Dienst mit 53 Tools. 15 davon decken die 18 offiziellen ab; die übrigen **38 haben beim offiziellen Server keine Entsprechung**. Sie bringen vor allem drei Bereiche hinzu, die dort vollständig fehlen — **Traceability-Auswertung**, **Test-Management** und **Baseline-Vergleiche** — dazu Massenoperationen, Anhänge, Reviews sowie eine eigene Verwaltung von Zugängen, Nutzung und Nachweisen.

Der entscheidende Unterschied liegt nicht in der Anzahl der Tools, sondern in der Verantwortung: Beim offiziellen Server trägt Jama Betrieb, Pflege und Kompatibilität. Beim Eigenbau tragen wir sie — einschließlich der Aufbewahrung der Jama-Zugangsdaten.

Wichtige Einschränkung dieses Vergleichs: Der eigene Server ist bislang gegen Fixtures getestet, nicht gegen eine reale Jama-Instanz. Die Aussagen zum offiziellen Server stammen ausschließlich aus dessen öffentlicher Dokumentation; er wurde nicht selbst betrieben. Beide Punkte begrenzen die Belastbarkeit der Gegenüberstellung und sind vor einer Festlegung auszuräumen.

Rahmenbedingungen

Merkmal	Offizieller Jama Connect MCP	jama-mcp-service
Betrieb	Von Jama gehostet, Teil der Instanz	Selbst betrieben (Docker)
Endpunkt	<code><tenant-url>/mcp/mcp-core</code>	<code><eigene-url>/mcp</code>
Freischaltung	Muss von Jama je Tenant aktiviert werden (über Customer Success Manager)	Keine Freischaltung nötig, nutzt die reguläre REST-API
Zusätzliche Voraussetzung	Benutzer muss unter Admin → REST API → Managed Access Control freigegeben sein	Named-Creator-Lizenz (gilt für jeden REST-Zugriff)
Anmeldung	Eigens erzeugte MCP-OAuth-Zugangsdaten; bestehende API-Zugangsdaten funktionieren nicht	Jama-OAuth oder Basic Auth, verschlüsselt im Dienst hinterlegt
Client-Anbindung	Über <code>mcp-remote</code> als Zwischenschicht	Streamable HTTP direkt, alternativ stdio
Mindestversion	Jama Connect 9.35	Keine, solange die REST-API v1 verfügbar ist
Support	Durch Jama	Keiner
Kosten	Teil des Jama-Vertrags, Konditionen instanzabhängig	Betriebskosten des eigenen Servers, Entwicklungs- und Pflegeaufwand

Funktionsumfang im Überblick

Die Zeilen folgen den Toolsets des Eigenbaus; die offiziellen Tools sind ihnen sachlich zugeordnet.

Bereich	Offiziell	Eigen
Projekte, Schema, Suche, Navigation	5	11
Beziehungen und Traceability (lesend)	1	5
Schreibende Operationen auf Items	7	12
Kommentare und Workflow	3	4
Test-Management	0	7
Baselines und Historie	2	6
Reviews	0	3
Anhänge und Reports	0	5
Tools gesamt	18	53
davon schreibend	10	21
davon mit Bestätigungspflicht	nicht dokumentiert	3
Vorgefertigte Abläufe (MCP Prompts)	nicht dokumentiert	5
MCP Resources	nicht dokumentiert	2

Abdeckung der offiziellen Tools

Alle 18 Tools des offiziellen Servers haben eine Entsprechung im Eigenbau.

Offizielles Tool	Entsprechung im Eigenbau	Anmerkung
<code>create_jama_item</code>	<code>jama_create_item</code>	Gleichwertig. Der Eigenbau löst Picklist-Werte aus Klartext auf
<code>edit_jama_entity</code>	<code>jama_update_item</code>	Gleichwertig
<code>move_jama_entity</code>	<code>jama_move_item</code>	Gleichwertig
<code>get_jama_entity_details</code>	<code>jama_get_item</code>	Der Eigenbau wandelt Rich-Text nach Markdown
<code>get_jama_entity_type_details</code>	<code>jama_get_project_schema</code>	Der Eigenbau liefert zusätzlich alle Picklist-Werte und Beziehungstypen in einem Aufruf
<code>create_jama_comment</code>	<code>jama_add_comment</code>	Gleichwertig
<code>create_jama_component</code>	<code>jama_create_container (kind: component)</code>	Drei offizielle Tools in einem zusammengefasst
<code>create_jama_set</code>	<code>jama_create_container (kind: set)</code>	siehe oben
<code>create_jama_folder</code>	<code>jama_create_container (kind: folder)</code>	siehe oben
<code>list_jama_projects</code>	<code>jama_list_projects</code>	Gleichwertig, im Eigenbau zwischengespeichert
<code>get_jama_project_details</code>	teilweise in <code>jama_list_projects</code>	Lücke: kein eigenes Detail-Tool je Projekt
<code>create_jama_entity_relationship</code>	<code>jama_create_relationship</code>	Gleichwertig
<code>list_jama_entity_relationships</code>	<code>jama_get_relationships</code>	Der Eigenbau kennzeichnet zusätzlich Suspect-Beziehungen
<code>create_jama_baseline</code>	<code>jama_create_baseline</code>	Gleichwertig

Offizielles Tool	Entsprechung im Eigenbau	Anmerkung
<code>list_jama_baseline</code>	<code>jama_list_baselines</code>	Gleichwertig
<code>execute_jama_item_workflow_transition</code>	<code>jama_execute_workflow_transition</code>	Gleichwertig
<code>list_jama_item_workflow_transition_options</code>	<code>jama_get_workflow_options</code>	Gleichwertig
<code>search_jama_entities</code>	<code>jama_search_items</code>	Beide textbasiert; der Eigenbau reicht Lucene-Syntax durch

Die 38 zusätzlichen Tools

15 Tools des Eigenbaus decken die 18 offiziellen ab. Die übrigen **38 haben beim offiziellen Server keine Entsprechung**. Nachfolgend vollständig, nach Toolset.

Kern — 7 Tools

Tool	Zweck
<code>jama_whoami</code>	Verbindung, angemeldeter Benutzer, Lizenztyp und verfügbare API-Versionen prüfen
<code>jama_get_items_batch</code>	Bis zu 50 Items in einem Tool-Aufruf. Spart gegenüber Einzelabrufen sowohl Rate-Limit-Budget als auch Kontext
<code>jama_browse_tree</code>	Projektstruktur über mehrere Ebenen erkunden, ohne alle Items zu laden
<code>jama_run_filter</code>	In der Jama-Oberfläche gespeicherte Filter ausführen. Nutzt die von Fachanwendern gepflegten Bedingungen, die sich über eine Textsuche kaum nachbauen ließen
<code>jama_list_releases</code>	Releases mit Datum und Aktivstatus
<code>jama_list_tags</code>	Tags eines Projekts, wahlweise die damit versehenen Items
<code>jama_list_users</code>	Benutzer auflisten, um Zuweisungen und Verantwortliche aufzulösen

Traceability — 4 Tools

Der inhaltlich größte Unterschied. Der offizielle Server kann Beziehungen auflisten und anlegen; auswerten kann er sie nicht.

Tool	Zweck
<code>jama_trace_chain</code>	Verfolgt Verknüpfungsketten über mehrere Ebenen, etwa von der Stakeholder-Anforderung bis zum Testfall. Mit Zyklenerkennung und Knotenbegrenzung. Ersetzt Dutzende Einzelaufrufe
<code>jama_find_trace_gaps</code>	Findet Items ohne geforderte Verknüpfung — Anforderungen ohne Testfall, Sicherheitsanforderungen ohne Verifikation. Das Werkzeug für Nachweise nach ASPICE, ISO 26262 oder IEC 62304
<code>jama_trace_matrix</code>	Zuordnungstabelle zwischen zwei ItemTypes samt Abdeckungsquote
<code>jama_check_relationship_rules</code>	Prüft im Trockenlauf, ob eine geplante Verknüpfung dem Regelwerk entspricht — bevor Jama sie mit einem wenig aussagekräftigen 400 ablehnt

Schreibende Operationen — 7 Tools

Tool	Zweck
<code>jama_bulk_create_items</code>	Mehrere Items nacheinander anlegen, etwa aus einer Anforderungsliste. Trockenlauf standardmäßig aktiv, Teilerfolgs-Bericht
<code>jama_bulk_update_items</code>	Dieselben Feldwerte auf vielen Items setzen. Trockenlauf standardmäßig aktiv
<code>jama_delete_item</code>	Item löschen. Erfordert ausdrückliche Bestätigung
<code>jama_duplicate_item</code>	Kopie als Grundlage für eine Variante
<code>jama_delete_relationship</code>	Verknüpfung entfernen. Erfordert Bestätigung, weil dabei Nachweisketten zerstört werden können
<code>jama_manage_tags</code>	Tags setzen und entfernen, ohne Fachfelder anzufassen
<code>jama_lock_item</code>	Bearbeitungssperre lesen und setzen, damit nicht parallel in der Oberfläche gearbeitet wird

Zusammenarbeit — 1 Tool

Tool	Zweck
<code>jama_list_comments</code>	Kommentare samt Antworten lesen. Die dokumentierte Tool-Liste des offiziellen Servers enthält mit <code>create_jama_comment</code> nur ein schreibendes Kommentar-Tool; ob <code>get_jama_entity_details</code> Kommentare mitliefert, geht aus der Dokumentation nicht hervor

Test-Management — 7 Tools

Beim offiziellen Server vollständig abwesend.

Tool	Zweck
<code>jama_list_testplans</code>	Testpläne eines Projekts
<code>jama_create_testplan</code>	Testplan mit Testgruppen und zugeordneten Testfällen anlegen — fasst drei aufeinander aufbauende Jama-Aufrufe zusammen
<code>jama_list_testcycles</code>	Testzyklen eines Projekts oder Plans
<code>jama_create_testcycle</code>	Testzyklus anlegen; Jama erzeugt dabei die Testläufe
<code>jama_list_testruns</code>	Testläufe eines Zyklus, nach Status filterbar
<code>jama_update_testrun</code>	Ergebnis eintragen, inklusive Einzelergebnissen je Testschritt
<code>jama_testcycle_summary</code>	Zyklus auswerten: Verteilung, Fortschritt, Erfolgsquote und die fehlgeschlagenen Läufe mit Begründung

Historie — 4 Tools

Tool	Zweck
<code>jama_compare_baselines</code>	Feldweiser Unterschied zwischen zwei Ständen: hinzugekommen, entfallen, geändert. Jama liefert nur die beiden Bestandslisten — der Vergleich entsteht hier
<code>jama_get_item_history</code>	Versionshistorie mit Bearbeiter und Zeitpunkt, wahlweise zwei Versionen feldweise verglichen
<code>jama_get_activities</code>	Aktivitätsstrom eines Projekts oder Items, alternativ die administrativen Vorgänge der Instanz
<code>jama_restore_deleted</code>	Gelöschte Items wiederherstellen. Erfordert Bestätigung

Reviews — 3 Tools

Nutzen `labs` -Endpunkte (ab Jama Connect 9.32).

Tool	Zweck
<code>jama_list_reviews</code>	Reviews eines Projekts mit Status und Organisator
<code>jama_get_review_status</code>	Fortschritt einer Revision: wer hat abgestimmt, wer steht noch aus
<code>jama_list_review_comments</code>	Alle Kommentare eines Reviews, als Grundlage für eine thematische Bündelung

Dateien und Reports — 5 Tools

Tool	Zweck
<code>jama_list_attachments</code>	Anhänge eines Items
<code>jama_upload_attachment</code>	Anhang hochladen und verknüpfen — fasst drei Jama-Aufrufe zusammen
<code>jama_download_attachment</code>	Anhang herunterladen; Text direkt, Binärdaten als base64, mit Größenbegrenzung
<code>jama_list_reports</code>	In Jama hinterlegte Reports (<code>labs</code> , ab 8.79)
<code>jama_run_report</code>	Report starten (asynchron, <code>labs</code>)

Dazu ohne Entsprechung

- **5 vorgefertigte Abläufe (MCP Prompts):** Anforderungen fachlich prüfen, Traceability-Lückenanalyse, Testplan aus Anforderungen ableiten, Baseline-Änderungsbericht, Spezifikationstext in Items zerlegen.
- **2 MCP Resources:** Projektliste und die Fähigkeiten des jeweiligen Zugangs.

Was der Eigenbau darüber hinaus leistet

Von Jama ausdrücklich ausgeschlossene Fähigkeiten

Die offizielle Dokumentation nennt sieben Einschränkungen. Vier davon adressiert der Eigenbau bewusst:

Offizielle Einschränkung	Umgang im Eigenbau
„No bulk transactional operations“	<code>jama_bulk_create_items</code> und <code>jama_bulk_update_items</code> mit verpflichtendem Trockenlauf. Keine echte Transaktion, aber ein Teilerfolgs-Bericht, der benennt, was angelegt wurde und was nicht
„No multi-call atomic transactions“	Ebenfalls keine Atomarität. Mehrstufige Abläufe wie <code>jama_create_testplan</code> oder <code>jama_upload_attachment</code> fassen die Aufrufe zusammen und melden Teilfehler ausdrücklich
„No bulk export; MCP is intended for retrieval of scoped context only“	<code>jama_trace_matrix</code> und <code>jama_testcycle_summary</code> liefern Auswertungen über größere Bestände, allerdings als verdichtete Kennzahlen statt als Rohdaten-Export
„No semantic search; search is text-based only“	Nicht adressiert. Der Eigenbau sucht ebenfalls rein textbasiert
„No webhook or event subscriptions“	Nicht adressiert
„No streaming tool calls“	Nicht adressiert
„No automatic relationship rule creation“	Nicht adressiert. <code>jama_check_relationship_rules</code> prüft die Regeln lediglich vorab, statt sie anzulegen

Aufbereitung der Antworten

Kein Tool, aber für die Nutzbarkeit entscheidend. Der offizielle Server macht dazu keine Angaben; im Eigenbau durchläuft jede Antwort dieselbe Aufbereitung:

- **Rich-Text nach Markdown.** Jama liefert Beschreibungen als HTML. Roh weitergereicht kostet ein einziges Requirement schnell mehrere tausend Token, der Großteil davon Markup.
- **Picklist-Werte in Klartext.** Aus `"status": 307` wird `"status": "Approved"`. Ohne das müsste ein Sprachmodell raten oder zusätzliche Aufrufe machen.
- **Eingebettete Bilder als Platzhalter.** Data-URLs in Beschreibungen erreichen sechsstellige Zeichenzahlen und würden das Kontextfenster allein füllen.
- **Antwortbudget je Aufruf.** Zu lange Antworten werden gekürzt — mit einem ausdrücklichen Hinweis, damit das Modell den Rumpf nicht für vollständig hält.
- **Sprechende Fehlermeldungen.** Ein 400 erklärt, dass Custom Fields ein Suffix wie `feld$32` tragen und zuerst das Schema abzurufen ist. Ein nacktes „400“ führt sonst zu Wiederholungsschleifen.

Betrieb und Steuerung

- **Eigene API-Key-Verwaltung** mit Rotation, Ablaufdatum und Deaktivierung.
 - **Toolsets je Zugang:** Ein Key sieht nur die freigeschalteten Tools. Das begrenzt sowohl den Schaden eines kompromittierten Zugangs als auch die Kontextlast im Sprachmodell.
 - **Projekt-Allowlist** je Zugang, zusätzlich zu den Jama-Berechtigungen.
 - **Globale Notbremse**, die alle schreibenden Tools sofort sperrt.
 - **Nutzungsauswertung** mit Latenzen, Fehlerquoten, Cache-Trefferquote und geschätztem Token-Verbrauch je Tool.
 - **Eigener Audit-Trail** über alle schreibenden Operationen, als CSV exportierbar.
 - **Rate-Limit-Steuerung:** ein Token-Bucket je Jama-Verbindung, der bewusst unter Jamas Grenze von 10 Anfragen pro Sekunde bleibt, damit andere Integrationen der Instanz nicht ausgebremst werden.
 - **Antwort-Aufbereitung:** Rich-Text wird nach Markdown gewandelt, Picklist-Werte in Klartext aufgelöst, eingebettete Bilder ersetzt und Antworten auf ein Token-Budget begrenzt.
-

Was für den offiziellen Server spricht

Diese Punkte wiegen schwer und lassen sich durch Funktionsumfang nicht aufwiegen.

Verantwortung für die Zugangsdaten. Beim offiziellen Server bleiben die Jama-Zugangsdaten bei Jama. Der Eigenbau speichert sie — verschlüsselt, aber eben bei uns. Das verlagert Haftung und Sorgfaltspflicht.

Pflege bei API-Änderungen. Ändert Jama die REST-API, passt Jama den eigenen MCP-Server an. Beim Eigenbau müssen wir nachziehen. Das betrifft besonders die `labs`-Endpunkte für Reviews und Reports, die Jama ausdrücklich ohne Supportzusage anbietet.

Kein eigener Betrieb. Der offizielle Server benötigt weder Server, noch Datenbank, noch Updates, noch Sicherheitspflege.

Support und Gewährleistung. Bei Problemen gibt es einen Ansprechpartner. Beim Eigenbau gibt es uns.

Geringere Angriffsfläche. Jeder selbst betriebene Dienst mit gespeicherten Zugangsdaten ist ein zusätzliches Ziel.

Vermutlich engere Integration. Die Dokumentation nennt „controlled tool calls“ und die Durchsetzung von Berechtigungen anhand der Rolle des Credential-Inhabers. Ob und wie die Aufrufe in Jamas eigenen Audit-Trail einfließen, geht aus der Dokumentation nicht hervor — das wäre vor einer Entscheidung zu klären.

Offene Risiken des Eigenbaus

Risiko	Bewertung	Umgang
Noch nicht gegen eine reale Jama-Instanz verifiziert	Hoch. Feldnamen, Custom-Field-Suffixe und die Verfügbarkeit der labs -Endpunkte sind ungeprüft	Zugang zu einer Test-Instanz beschaffen und die Contract-Tests ausführen, bevor produktiv gearbeitet wird
Pflegeaufwand bei Jama-Releases	Mittel	Contract-Tests gegen eine Test-Instanz nach jedem Jama-Release
Aufbewahrung der Zugangsdaten	Mittel	AES-256-GCM, Zugangsdaten je Person statt Sammelkonto, Rotation und Ablaufdatum
labs -Endpunkte ohne Supportzusage	Mittel	In eigene Toolsets ausgelagert, die einzeln abschaltbar sind; die Tools erkennen fehlende Endpunkte und melden das verständlich
Prompt Injection über Jama-Inhalte	Mittel	Antworten werden ausdrücklich als Daten gekennzeichnet; schreibende Tools verlangen eine Bestätigung im Client
Fehlbedienung durch das Sprachmodell	Mittel	Read-only als Standard, Toolsets je Zugang, Trockenlauf bei Massenänderungen, Bestätigungspflicht bei löschenden Tools, globale Notbremse

Einschätzung

Der offizielle Server ist die richtige Wahl, wenn der Bedarf im Abrufen und Bearbeiten einzelner Anforderungen liegt, die Instanz auf 9.35 oder höher läuft, die Freischaltung verfügbar ist und kein eigener Betrieb gewünscht wird. Das dürfte auf die Mehrzahl der Anwendungsfälle zutreffen.

Der Eigenbau lohnt sich, wenn mindestens einer dieser Punkte zutrifft:

- Test-Management, Baseline-Vergleiche oder Traceability-Analysen werden gebraucht — diese Bereiche deckt der offizielle Server gar nicht ab.
- Massenoperationen sind erforderlich, die der offizielle Server ausdrücklich ausschließt.
- Es wird eine eigene Sicht auf Nutzung, Kosten und Audit über mehrere Zugänge hinweg benötigt.
- Die Freischaltung durch Jama ist nicht verfügbar oder die Instanz ist älter als 9.35.
- Zugänge sollen fein granular je Person und Projekt geschnitten werden.

Ein Parallelbetrieb ist möglich und sinnvoll, weil beide Server unabhängig voneinander in einem MCP-Client eingetragen werden können. Der offizielle übernimmt dann das alltägliche Arbeiten an einzelnen Objekten, der Eigenbau die Auswertungen und Massenvorgänge, für die er gebaut wurde. Der Preis dafür ist, dass zwei Toolsätze im Kontextfenster liegen und das Sprachmodell zwischen ähnlichen Tools wählen muss.

Nächste Schritte

1. **Klären, ob der offizielle MCP für die Ziel-Instanz überhaupt freigeschaltet ist** und welche Konditionen daran hängen. Ohne diese Auskunft ist jede Entscheidung unvollständig.
2. **Zugang zu einer Test-Instanz beschaffen** und die Contract-Tests des Eigenbaus ausführen. Erst danach lässt sich sagen, ob er unter realen Bedingungen hält, was er verspricht.
3. **Beide Server nebeneinander an einem Client betreiben** und dieselben Aufgaben stellen. Der Tool-Katalog des Eigenbaus enthält dafür einen Probelauf, der die Antwort samt Token-Schätzung zeigt.
4. **Danach entscheiden** — nicht vorher.